



Penetration Testing

Cabal Labs (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test Scope	6
Security Rating	7
Severity Definitions	8
Status Definitions	9
Penetration Test Findings	10
Conclusion	17
Our Methodology	18
Disclaimers	20
About Hashlock	21

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Cabal Labs team partnered with Hashlock to conduct a security Penetration Test of their code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

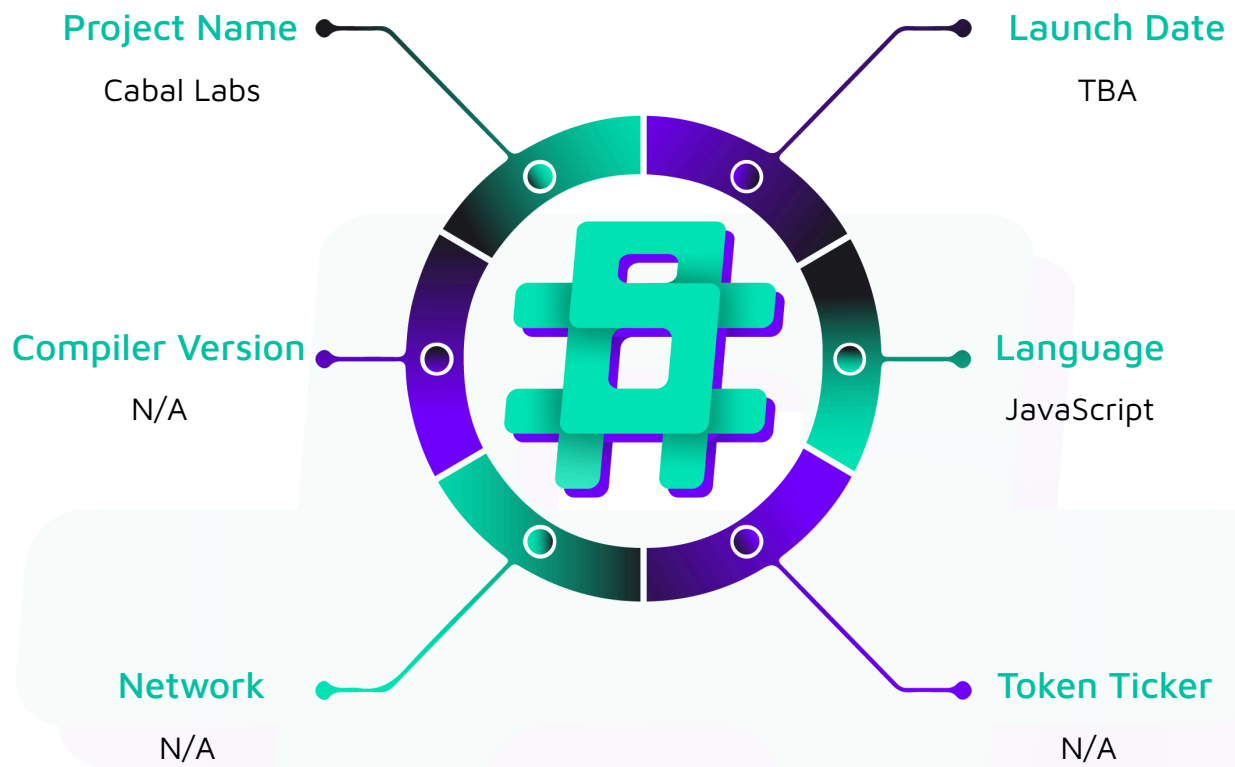
Cabal Labs is a DeFi dApp built on Solana Network that allows users to create new tokens and have a fair sale. These tokens can be swapped in and out for Solana native currency. Lastly, liquidity pools will be launched to Raydium.

Project Name: Cabal Labs

Project Type: Defi, dApp

Logo:



Visualised Context:

Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the Cabal Labs project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	Cabal Labs code base
Platform	Web App
Penetration Test Date	May, 2025
Repository	https://github.com/cabalfun/cabal-frontend
GitHub Commit Hash (If applicable)	bd90f2a6659599f61ef95ce9cfcd696c196da0de
Fix Review GitHub Commit Hash (If applicable)	ebcb3e6f4a3704e438c64c3e2077846a68369979

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 Medium-severity vulnerabilities

1 Low-severity vulnerabilities

5 QA

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

Medium

[M-01] packages/app/components/MemeSwapWidget/BuyForm.js#BuyForm

- Users cannot specify slippage tolerance parameters from the front-end

Description

When users attempt to buy or sell tokens, the slippage parameters (`minTokenAmountOut` and `minNativeTokenAmountOut` variables) are hardcoded to zero when dispatching swap messages to the program.

This is shown in the following instances:

- Line 67 in packages/app/components/MemeSwapWidget/BuyForm.js
- Line 67 in packages/app/components/MemeSwapWidget/SellForm.js
- Lines 162 and 230 in packages/app/components/MemeSwapWidget/utils.js
- Line 307 in packages/app/app/[chain]/new/_components/utils.js

This is problematic because users should be able to provide slippage parameters during the swap as part of the protocol's intended operation to ensure they receive at least the requested amount.

It is worth noting that the output amount (`amountOut` variable) is queried and shown to the user, but it is not enforced during the swap operation. For example, see lines 55 to 63 in packages/app/components/MemeSwapWidget/BuyForm.js.

Impact

Users cannot supply slippage tolerance parameters as intended, potentially exposing them to sandwich attacks and causing them to lose funds.

Recommendation

Consider modifying the implementations to allow users to specify slippage tolerance parameters and enforce them within the front-end. Alternatively, consider



automatically setting the slippage tolerance parameters to the estimated output amount (`amountOut` variable) to ensure users receive the amount shown in the front-end.

Status

Acknowledged



Low

[L-01]

packages/app/middlewares/httpBasicAuthenticationMiddleware.js#authenticate - Environment variable should be used to avoid exposing confidential information

Description

In the following instances, the hardcoded values should be configured as environment variables:

- In line 8, the username and password in `packages/app/middlewares/httpBasicAuthenticationMiddleware.js` are hardcoded.
- In line 4 of `packages/ui/src/components/Giphy.js`, the Giphy API key is hardcoded.

Impact

Users who have read access to the front-end can retrieve the username, password, and Giphy API key, potentially exposing confidential information.

Recommendation

Consider defining the hardcoded values as environmental variables.

Status

Resolved

QA

[Q-01]

`packages/app/app/[chain]/[tokenAddress]/_components/MemeTokenStats.js`

- Import statements can be combined

Description

In the following instances, the import statements can be combined to reduce code redundancy:

- Lines 2 and 3 in `packages/app/app/[chain]/[tokenAddress]/_components/MemeTokenStats.js`
- Lines 1 and 2 in `packages/app/app/[chain]/_components/HotMemeTokenItem.js`
- Lines 1 and 2 in `packages/app/app/[chain]/_components/MemeTokenItem.js`

Recommendation

Consider grouping the imports into a single line.

Status

Resolved

[Q-02]packages/app/app/[chain]/new/_components/NewMemeTokenForm.js#getButtonLoadingText - Switch cases can be combined

Description

In the following instances, the cases in the switch statements can be combined to reduce code redundancy:

- Lines 77 to 81 in packages/app/app/[chain]/new/_components/NewMemeTokenForm.js
- Lines 83 to 87 in packages/app/app/[chain]/new/_components/NewMemeTokenForm.js
- Lines 134 to 138 in packages/app/components/MemeSwapWidget/BuyForm.js
- Lines 140 to 144 in packages/app/components/MemeSwapWidget/BuyForm.js
- Lines 134 to 138 in packages/app/components/MemeSwapWidget/SellForm.js
- Lines 140 to 144 in packages/app/components/MemeSwapWidget/SellForm.js

Recommendation

Consider combining the cases to reduce code redundancy.

Status

Acknowledged

[Q-03] packages/app/components/MemeSwapWidget/BuyForm.js#handleRenderAmountError - Spelling error

Description

In the following instances, a spelling error of “Amount is to low/high” is found:

- Lines 174 and 190 in packages/app/app/[chain]/new/_components/MemeTokenPoolFieldset.js
- Lines 102 and 118 in packages/app/components/MemeSwapWidget/BuyForm.js
- Lines 102 and 118 in packages/app/components/MemeSwapWidget/SellForm.js

Recommendation

Consider replacing the text with “Amount is too low/high.”

Status

Resolved

[Q-04] packages/app/components/MemeSwapWidget/utils.js#getBuyOptions - Randoming from zero does not present use cases

Description

If the user chooses “*DEGEN*” mode, the amount of tokens to buy is randomized from zero to the user’s balance. However, if zero is chosen, its value does not present as a valid purchase option.

This is found in the following instances:

- Line 33 in packages/app/components/MemeSwapWidget/utils.js
- Line 33 in packages/app/components/RaydiumMemeSwapWidget/utils.js

Recommendation

Consider starting the randomization from one or a default minimum value defined in a constant.

Status

Acknowledged

[Q-05] packages/app/operations/mutations/useRaydiumBuy.js#useMutation

- Slippage values can be defined as constants

Description

In line 49 of `packages/app/operations/mutations/useRaydiumBuy.js` and `packages/app/operations/mutations/useRaydiumSell.js`, the slippage values of `0.001` can be defined as a constant to ease updates of the slippage parameter.

Recommendation

Consider defining the slippage values as a constant and utilizing them in the above instances.

Status

Resolved

Conclusion

After Hashlock's analysis, the Cabal Labs project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.